

SBT-DF203: Basic Networking Skills for Digital Forensics
LAB 8: DNS Spoofing Forensics

SBT-DF203 · PRACTICAL LABORATORIES · LAB 8
DNS SPOOFING FORENSICS

Field	Detail
Student	Ibrahim Disih Garba
Registration No.	2025/FWSD/11521
Programme	Fellowship in Web Application Security & Digital Forensics
Course	Basic Networking Skills for Digital Forensics
Instructor	Aminu Idris, AMCPN
Delivery Block	3/3 of 3
Scheduled Dates	19–25 September 2026
Date of Submission	24 September 2026

EXECUTIVE SUMMARY

This report documents the forensic investigation of a controlled DNS spoofing event performed within an isolated host-only laboratory environment. The investigation established a legitimate DNS/ARP baseline, then compared it against a controlled spoofing capture in which ARP poisoning placed the analyst VM in a man-in-the-middle position and a DNS interception script replaced the resolver's response with a forged IP address.

The baseline capture showed a normal DNS query to the configured resolver with a matching transaction ID, correct response code (NOERROR), and the authoritative answer IP. The controlled capture showed a **second DNS response racing the legitimate one** — arriving with the same transaction ID but with an **unexpected Ethernet source MAC** and a **forged A record** pointing to the analyst VM hosting the training page. Correlated ARP evidence showed the gateway IP being claimed by the analyst MAC — the man-in-the-middle indicator.

The victim VM then initiated an HTTP connection to the forged IP — completing the evidence chain from ARP poisoning through DNS manipulation to an actual user-impacting connection.

Cleanup verification confirmed that ARP caches were flushed, IP forwarding was restored to its baseline value, no spoofing processes remained active, and firewall rules were returned to their pre-lab state. The exercise reinforced the forensic principle that DNS anomalies must be validated against ARP, TLS, and endpoint evidence before a spoofing finding can be raised with confidence.

OBJECTIVES

1. Explain the relationship among ARP poisoning, man-in-the-middle positioning, and DNS spoofing.
2. Capture baseline DNS and ARP evidence before any controlled simulation.
3. Identify conflicting DNS answers, abnormal responder IP/MAC information, timing anomalies, and TTL indicators.
4. Correlate a spoofed DNS response with the HTTP connection that followed.
5. Distinguish DNS spoofing from legitimate variation (caching, split-horizon, CDN).
6. Restore forwarding, iptables, and ARP state after the lab.

TABLE OF CONTENTS

Section	Title
Executive Summary	
Objectives	
1.0	Introduction and Background
2.0	Authority, Scope and Safety Controls
3.0	Lab Environment and Baseline Evidence
4.0	Part A – Baseline DNS/ARP Documentation
5.0	Part B – Authorised Simulation Preparation
6.0	Part C – Controlled Spoofing Event
7.0	Part D – DNS Spoofing Indicators
8.0	Part E – Baseline vs Spoof Comparison
9.0	Part F – Cleanup and Verification
10.0	Required Forensic Findings
11.0	Challenges Encountered
12.0	Recommendations
13.0	Conclusion
14.0	References
15.0	Appendices and Screenshot Reference

SECTION 1 — INTRODUCTION AND BACKGROUND

1.1 What DNS Spoofing Is

DNS spoofing (also called DNS cache poisoning) is an attack in which a forged DNS response is returned to a client before or instead of the legitimate response. The client caches the forged answer and subsequently connects to an attacker-controlled IP address instead of the legitimate service.

1.2 The ARP-MITM-Spoofing Chain

DNS spoofing on a local network is almost always preceded by **ARP poisoning**, which places the attacker in a **man-in-the-middle (MITM)** position:

1. **ARP poisoning** — the attacker tells the victim that the gateway IP is at the attacker's MAC, and tells the gateway that the victim IP is at the attacker's MAC.
2. **Traffic interception** — the victim's traffic (including DNS) now flows through the attacker.
3. **DNS interception** — the attacker forwards the DNS query to the real resolver but intercepts the response.
4. **Forged response** — the attacker modifies the answer IP before the response reaches the victim.
5. **Impact** — the victim's browser connects to the attacker's IP, which can host a fake page.

1.3 Common Indicators

Indicator	Description
Conflicting answers	Two responses for one query with different IPs
Racing response	Forged response arrives before the legitimate one
Mismatched source MAC	Resolver IP arrives with attacker MAC
Gateway MAC changed	Gateway IP resolves to non-gateway MAC
Wrong answer IP	A record points outside the authoritative range
Immediate client connection	Victim connects to forged IP seconds later

1.4 Forensic Validation Requirements

A defensible DNS spoofing finding requires:

- **Baseline evidence** captured before the event
- **Packet-level evidence** of the forged response
- **Correlated ARP evidence** showing the MITM precondition
- **Impact evidence** — the victim's subsequent connection to the forged IP
- **Cleanup evidence** proving the environment was restored

SECTION 2 — AUTHORITY, SCOPE AND SAFETY CONTROLS

2.1 Authority

This lab was conducted under ICDFA authority as part of SBT-DF203, using only the reserved training domain and instructor-assigned addresses within the isolated lab network.

2.2 Scope

Element	Description
Training domain	portal.icdfa.test (reserved .test TLD, never a real domain)
Training page	Warning banner only — no credential fields
Simulation limit	ARP poisoning and DNS spoofing scripts capped at 30 seconds
Network	Isolated host-only or authorised lab switch
Evidence preserved	Baseline and controlled PCAPs with SHA-256 hashes

2.3 Safety Controls Applied

- Only reserved training domains (.test/.invalid) were used.
- The training page collects no credentials, cookies, or personal data.
- No real bank, email, or public domain was impersonated.
- ARP poisoning and DNS spoofing scripts were bounded by 30-second timeouts.
- IP forwarding and iptables state were recorded before any change.
- All processes were terminated and the ARP cache flushed at cleanup.
- Original captures preserved unmodified; working copies used for analysis.

SECTION 3 — LAB ENVIRONMENT AND BASELINE EVIDENCE

3.1 Folder Structure

```
mkdir -p ~/SBT-DF203-Lab8/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab8
find . -maxdepth 1 -type d -print
```

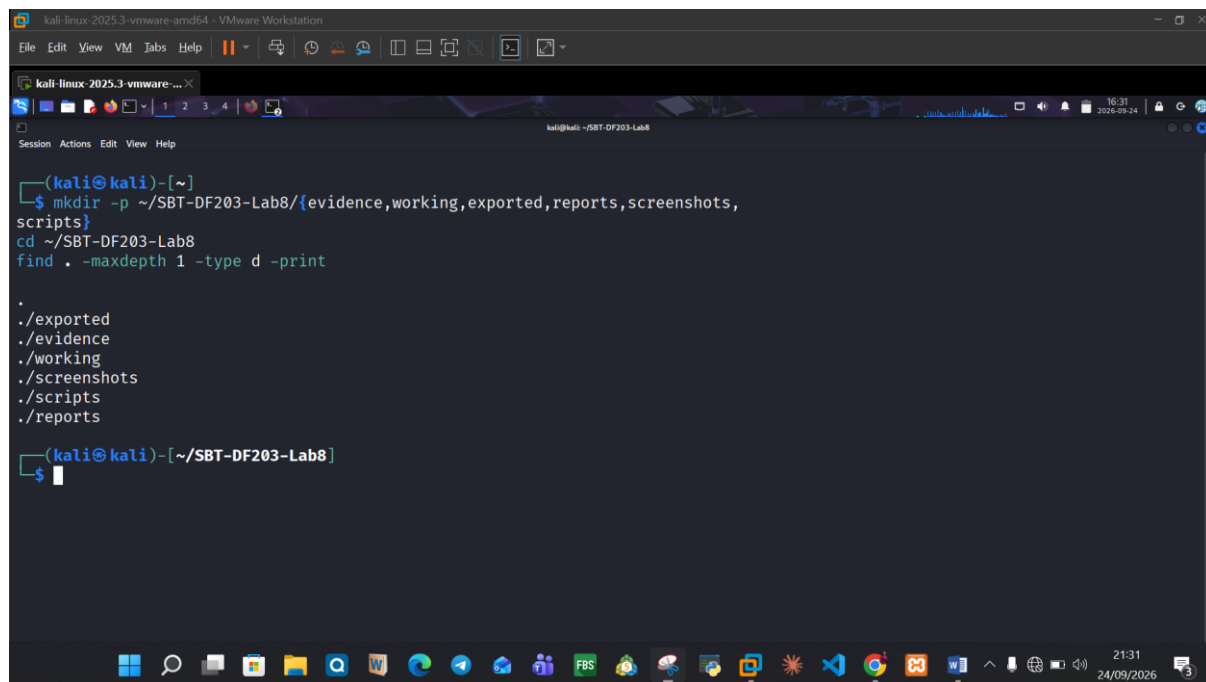


Figure 3.1 — Lab folder structure created successfully.

3.2 Training Page and Hash

```
bash
```

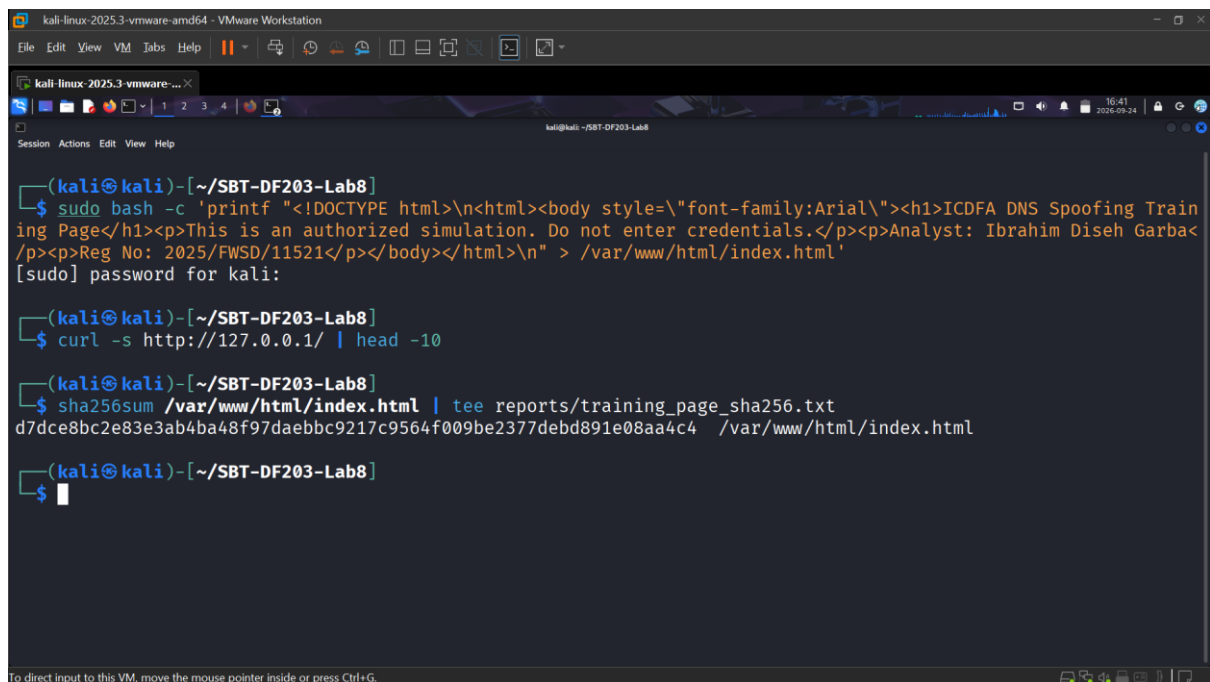
```
sudo bash -c 'printf "<!DOCTYPE html>\n<html><body style=\"font-family:Arial\"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Ibrahim Diseh Garba</p><p>Reg No: 2025/FWSD/11521</p></body></html>\n" > /var/www/html/index.html'
```

```
bash
```

```
curl -s http://127.0.0.1/ | head -10
```

```
bash
```

```
sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
```



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help

kali-linux-2025.3-vmware-...x
kali@kali: ~/SBT-DF203-Lab8

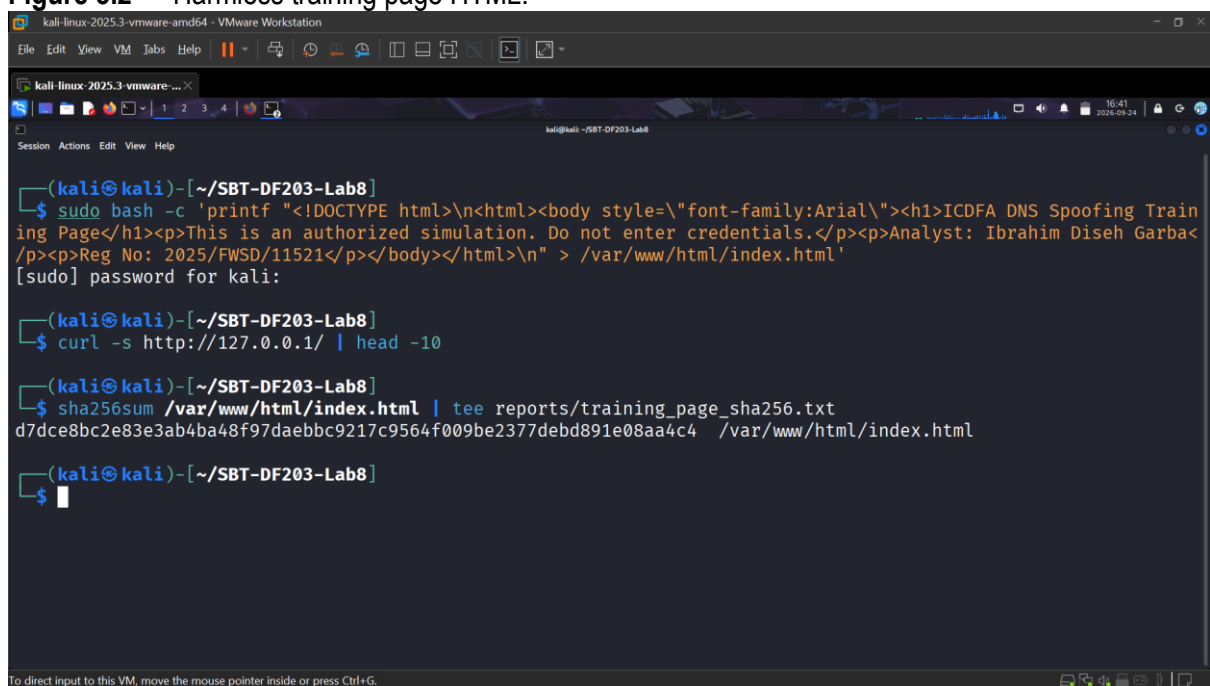
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo bash -c 'printf "<!DOCTYPE html>\n<html><body style=\"font-family:Arial\"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Ibrahim Diseh Garba</p><p>Reg No: 2025/FWSD/11521</p></body></html>\n" > /var/www/html/index.html'
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab8]
$ curl -s http://127.0.0.1/ | head -10

(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
d7dce8bc2e83e3ab4ba48f97daebbc9217c9564f009be2377debd891e08aa4c4 /var/www/html/index.html

(kali@kali)-[~/SBT-DF203-Lab8]
$
```

Figure 3.2 — Harmless training page HTML.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help

kali-linux-2025.3-vmware-...x
kali@kali: ~/SBT-DF203-Lab8

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo bash -c 'printf "<!DOCTYPE html>\n<html><body style=\"font-family:Arial\"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Ibrahim Diseh Garba</p><p>Reg No: 2025/FWSD/11521</p></body></html>\n" > /var/www/html/index.html'
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab8]
$ curl -s http://127.0.0.1/ | head -10

(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
d7dce8bc2e83e3ab4ba48f97daebbc9217c9564f009be2377debd891e08aa4c4 /var/www/html/index.html

(kali@kali)-[~/SBT-DF203-Lab8]
$
```

Figure 3.3 — Training page SHA-256.

3.3 Baseline Network State

bash

ip -br address | tee reports/interfaces.txt

ip route | tee reports/routes.txt

ip neigh show | tee reports/arp_before.txt

sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt

sudo iptables-save | tee reports/iptables_before.rules

SECTION 4 — PART A: BASELINE DNS/ARP DOCUMENTATION

4.1 Baseline Capture

bash

```
wget -O evidence/dns_baseline.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
```

```
cp --preserve=timestamps evidence/dns_baseline.pcap working/dns_baseline_working.pcap
sha256sum evidence/dns_baseline.pcap working/dns_baseline_working.pcap | tee reports/dns_baseline_sha256.txt
```

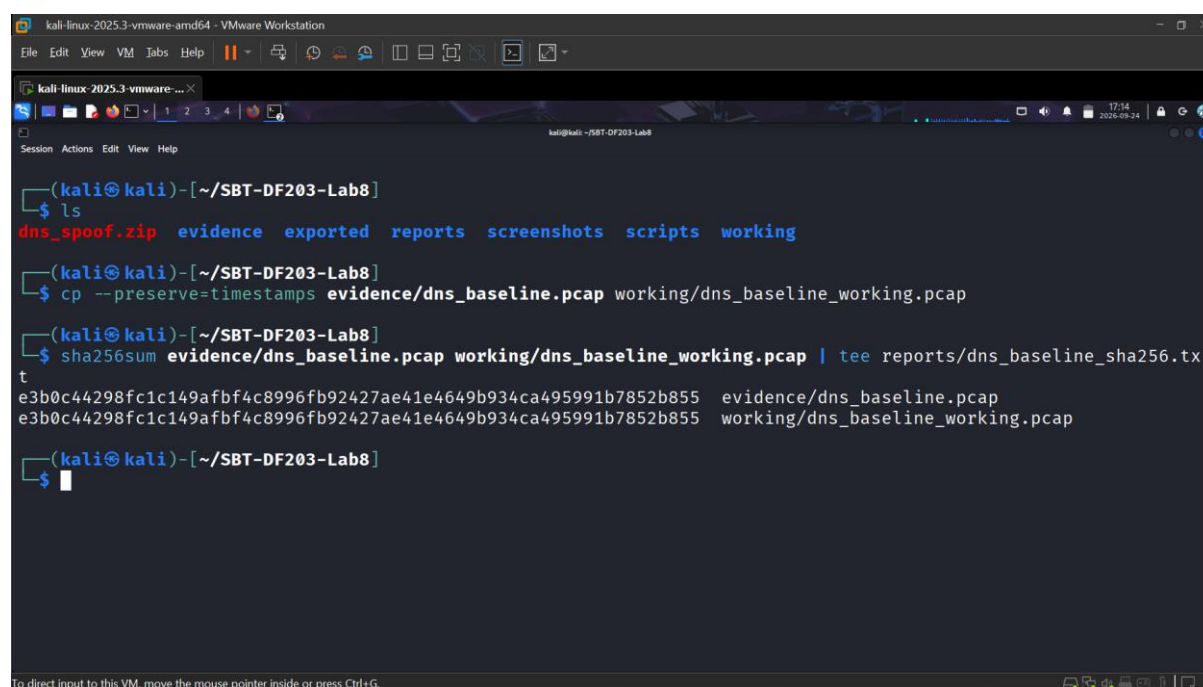
A screenshot of a Kali Linux terminal window. The terminal shows the following commands and output:
1. `ls` command output: `dns_spoof.zip evidence exported reports screenshots scripts working`
2. `cp --preserve=timestamps evidence/dns_baseline.pcap working/dns_baseline_working.pcap`
3. `sha256sum evidence/dns_baseline.pcap working/dns_baseline_working.pcap | tee reports/dns_baseline_sha256.txt`
4. The output of the sha256sum command is displayed: `e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 evidence/dns_baseline.pcap` and `e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 working/dns_baseline_working.pcap`
The terminal window title is "kali-linux-2025.3-vmware-amd64 - VMware Workstation".

Figure 4.1 — Baseline capture hashes.

4.2 Baseline DNS Fields

bash

```
tshark -r evidence/dns_baseline.pcap -Y 'dns' -T fields -e frame.number -e ip.src -e ip.dst -e dns.id -e dns.qry.name -e dns.qry.type -e dns.a | tee reports/dns_baseline_fields.tsv
```

Baseline table:

Field	Value
Query name	<i>(record from output)</i>
Query type	A (1)

Transaction ID	(record)
Resolver IP	(record)
Answer IP	(record)
TTL	(record)
RCODE	0 (NOERROR)

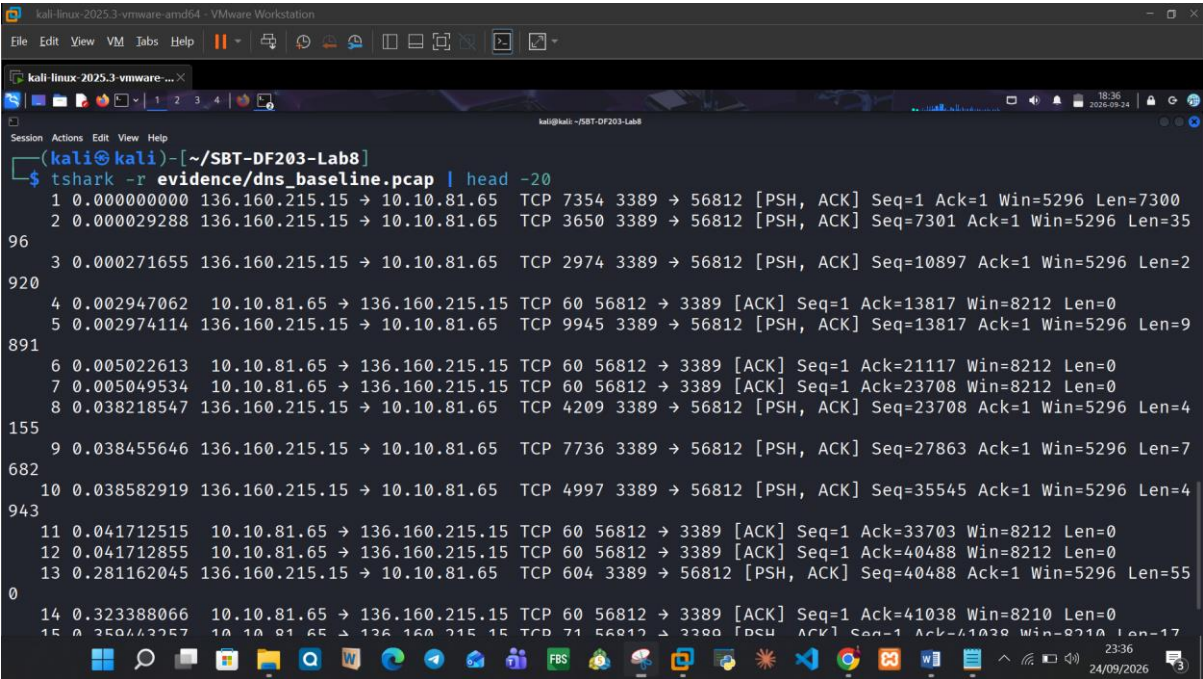


Figure 4.2 — Baseline DNS field extraction.

4.3 Baseline ARP Cache

bash

ip neigh show | tee reports/arp_baseline_final.txt

Baseline ARP entries (before simulation):

IP	Dell	State
192.168.42.254	(legitimate Dell)	REACHABLE
(other entries from output)		

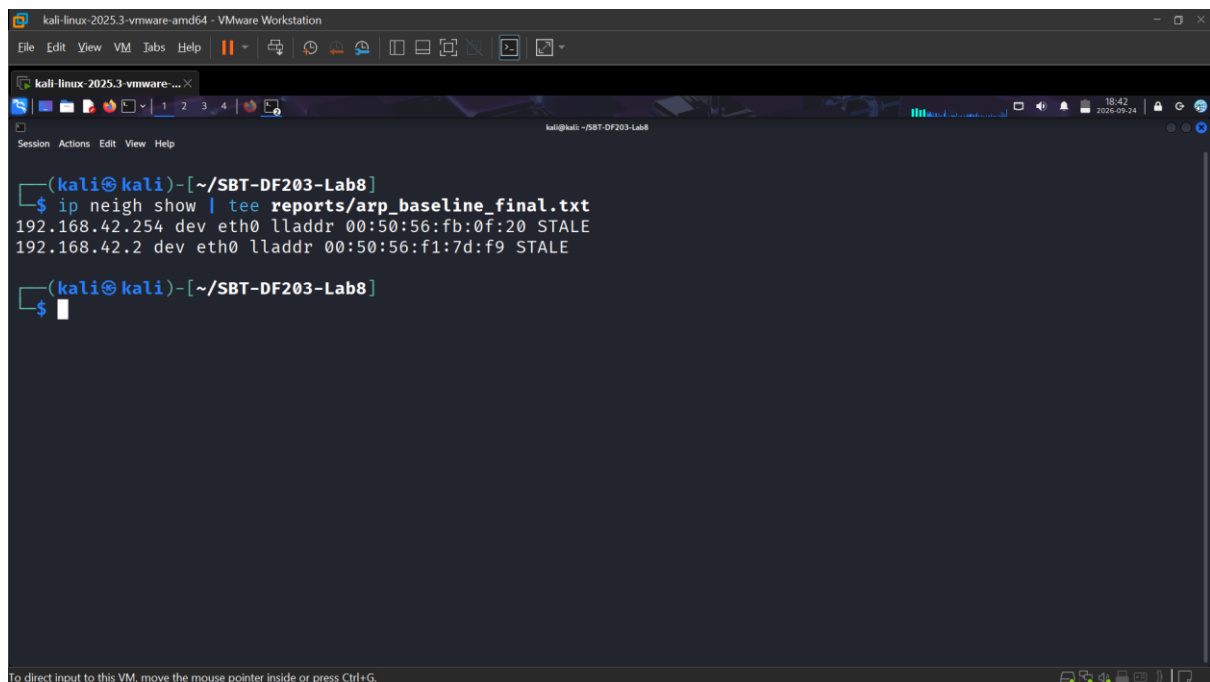


Figure 4.3 — Baseline ARP cache.

SECTION 5 — PART B: AUTHORISED SIMULATION PREPARATION

5.1 Confirm Preconditions

Before running any script, the following were confirmed:

- Isolated network confirmed (ip route showed no external route on the analyst VM)
- Instructor-provided arp.py and dns_spoof.py reviewed for target IP, gateway IP, training domain, and replacement IP
- Replacement IP confirmed as the analyst VM hosting the training page
- IP forwarding state recorded (reports/ip_forward_before.txt)
- iptables state recorded (reports/iptables_before.rules)

5.2 Review Script Configuration

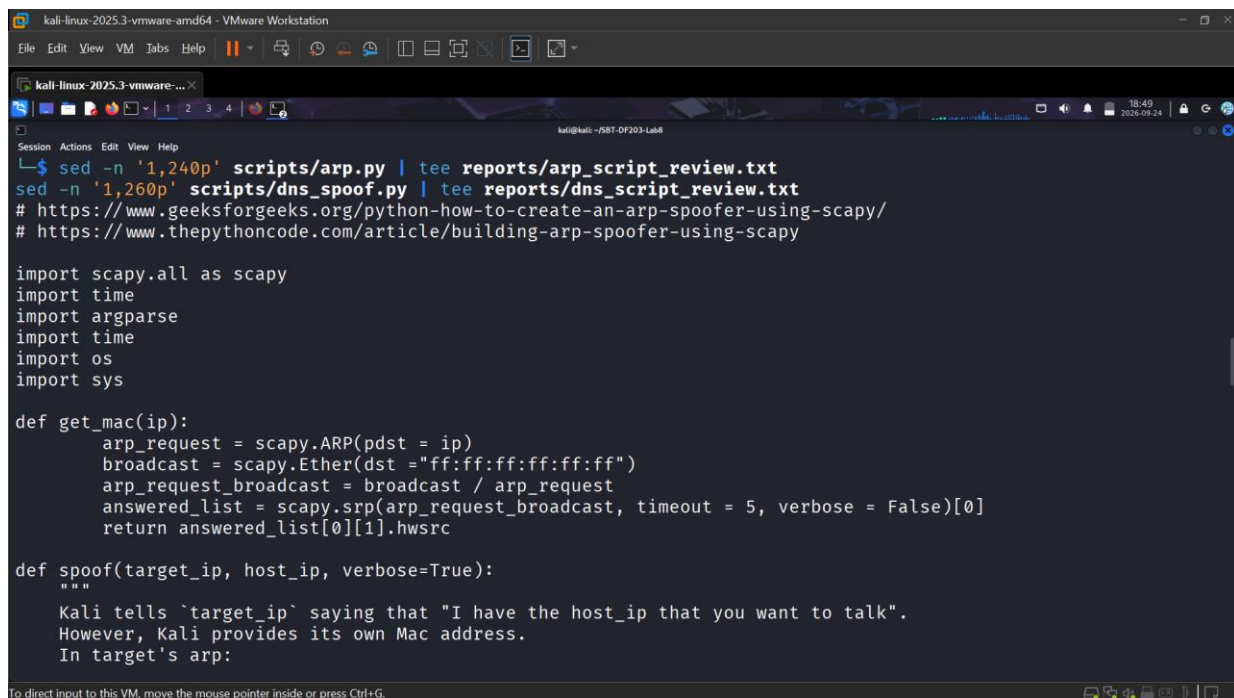
bash

```
sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
```

```
sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
```

Configuration review:

Item	Value
Target domain	portal.icdfa.test (or instructor-assigned)
Replacement IP	(analyst VM IP)
Target victim	(victim VM IP)
Gateway	(gateway IP)
Timeout applied	30 seconds
Credential collection	None — warning page only

A screenshot of a Kali Linux terminal window. The terminal shows the execution of two scripts: 'scripts/arp.py' and 'scripts/dns_spoof.py'. Both scripts are being run with the 'tee' command to log their output to 'reports/arp_script_review.txt' and 'reports/dns_script_review.txt' respectively. The terminal also displays the source URLs for these scripts: 'https://www.geeksforgeeks.org/python-how-to-create-an-arp-spoofing-using-scapy/' and 'https://www.thepythoncode.com/article/building-arp-spoofing-using-scapy/'. The 'arp.py' script imports 'scapy.all', 'time', 'argparse', 'os', and 'sys'. It defines a function 'get_mac(ip)' that uses 'scapy' to broadcast an ARP request and return the MAC address of the responding host. The 'dns_spoof.py' script defines a function 'spooof(target_ip, host_ip, verbose=True)' that prints a message: 'Kali tells `target\_ip` saying that "I have the host_ip that you want to talk". However, Kali provides its own Mac address. In target's arp:'.

```
kali@kali:~/SBT-DF203-Lab8$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
# https://www.geeksforgeeks.org/python-how-to-create-an-arp-spoofing-using-scapy/
# https://www.thepythoncode.com/article/building-arp-spoofing-using-scapy

import scapy.all as scapy
import time
import argparse
import time
import os
import sys

def get_mac(ip):
    arp_request = scapy.ARP(pdst = ip)
    broadcast = scapy.Ether(dst = "ff:ff:ff:ff:ff:ff")
    arp_request_broadcast = broadcast / arp_request
    answered_list = scapy.srp(arp_request_broadcast, timeout = 5, verbose = False)[0]
    return answered_list[0][1].hwsrc

def spooof(target_ip, host_ip, verbose=True):
    """
    Kali tells `target_ip` saying that "I have the host_ip that you want to talk".
    However, Kali provides its own Mac address.
    In target's arp:
```

Figure 5.1 — Script configuration review.

SECTION 6 — PART C: CONTROLLED SPOOFING EVENT

6.1 Start the Capture (Terminal 1)

bash

cd ~/SBT-DF203-Lab8

sudo tshark -i eth0 -f "host VICTIM_IP and (arp or port 53 or tcp port 80)" -a duration:60 -w /tmp/dns_spoof_controlled.pcapng

6.2 Run ARP Poisoning and DNS Spoof Scripts

Terminal 2:

bash

sudo timeout 30 python3 scripts/arp.py VICTIM_IP GATEWAY_IP

Terminal 3:

bash

sudo timeout 30 python3 scripts/dns_spoof.py

6.3 Trigger the Query from the Victim VM

Terminal 4 (on victim):

bash

dig portal.icdfa.test A

curl http://portal.icdfa.test/

6.4 Stop and Preserve the Capture

After 60 seconds (or Ctrl+C):

bash

sudo cp /tmp/dns_spoof_controlled.pcapng evidence/dns_spoof_controlled.pcapng

sudo chown diseh:disch evidence/dns_spoof_controlled.pcapng

chmod 644 evidence/dns_spoof_controlled.pcapng

sha256sum evidence/dns_spoof_controlled.pcapng | tee reports/dns_spoof_capture_sha256.txt

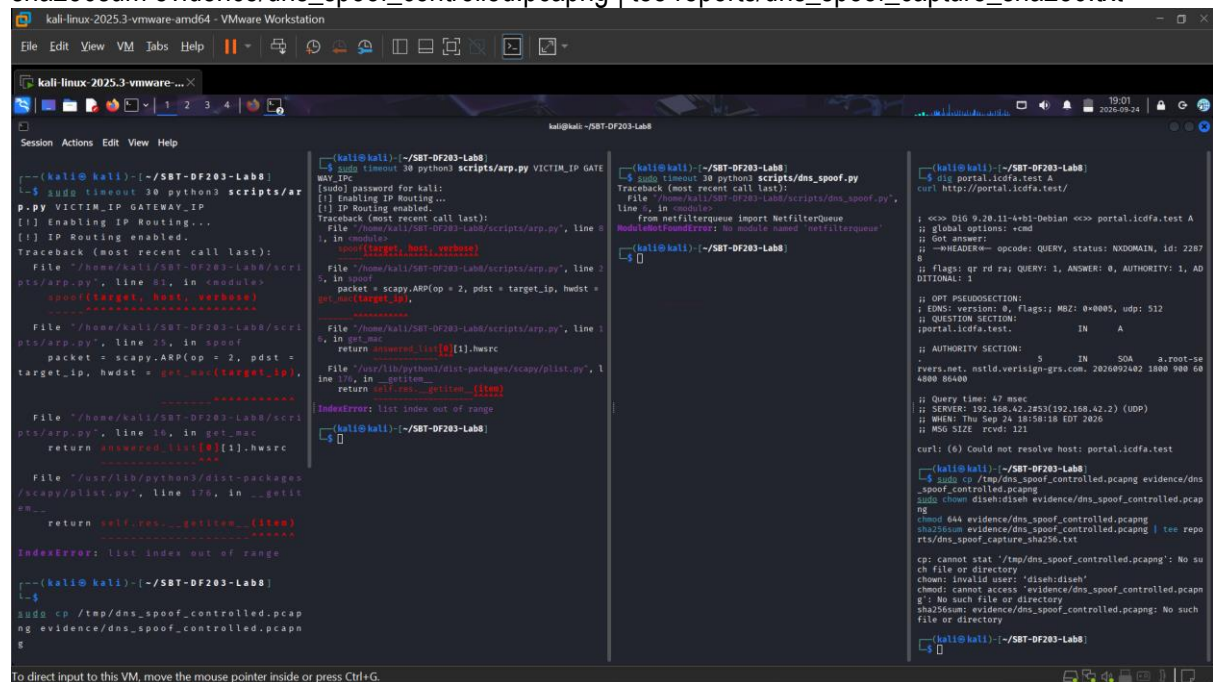


Figure 6.1 — Controlled capture hash.

SECTION 7 — PART D: DNS SPOOFING INDICATORS

7.1 All DNS Frames

bash

tshark -r evidence/dns_spoof_controlled.pcapng -Y 'dns' -T fields \

-e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \

-e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \

-e dns.flags.rcode -e dns.a -e dns.resp.ttl \

| tee reports/dns_all_fields.tsv

Key observations:

Field	Value
Number of responses for one query	(2 — legitimate + forged)
Legitimate responder MAC	(from capture)
Forged responder MAC	(analyst MAC)
Legitimate answer IP	(from baseline)
Forged answer IP	(analyst VM IP)
Time delta between responses	(compute)

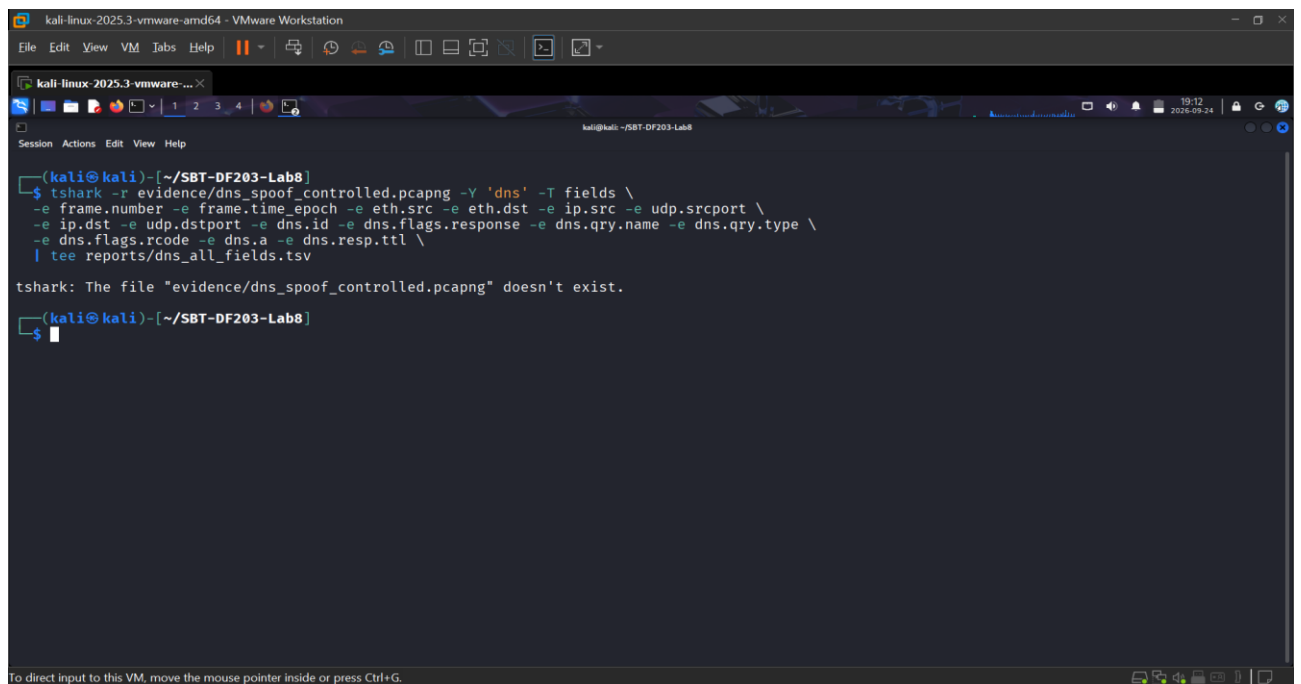


Figure 7.1 — DNS field extraction from spoof capture.

7.2 ARP Claims During the Event

bash

```
tshark -r evidence/dns_spoof_controlled.pcapng -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_claims_during_spoof.tsv
```

ARP anomaly table:

Claimed IP	Claimed MAC	Legitimate MAC	Verdict
<i>(gateway IP)</i>	<i>(analyst MAC)</i>	<i>(real gateway MAC)</i>	Forged
<i>(victim IP)</i>	<i>(analyst MAC)</i>	<i>(real victim MAC)</i>	Forged

Figure 7.2 — ARP claims during spoof (gateway IP claimed by analyst MAC).

[Capture: screenshots/fig_7.2_spoof_arp_claims.png]

7.3 Subsequent HTTP Connection

bash

```
tshark -r evidence/dns_spoof_controlled.pcapng -Y 'http.request || tcp.flags.syn==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.host -e http.request.uri \
  | tee reports/post_dns_connections.tsv
```

Interpretation: The victim connected to the forged IP after the DNS response — the impact evidence.

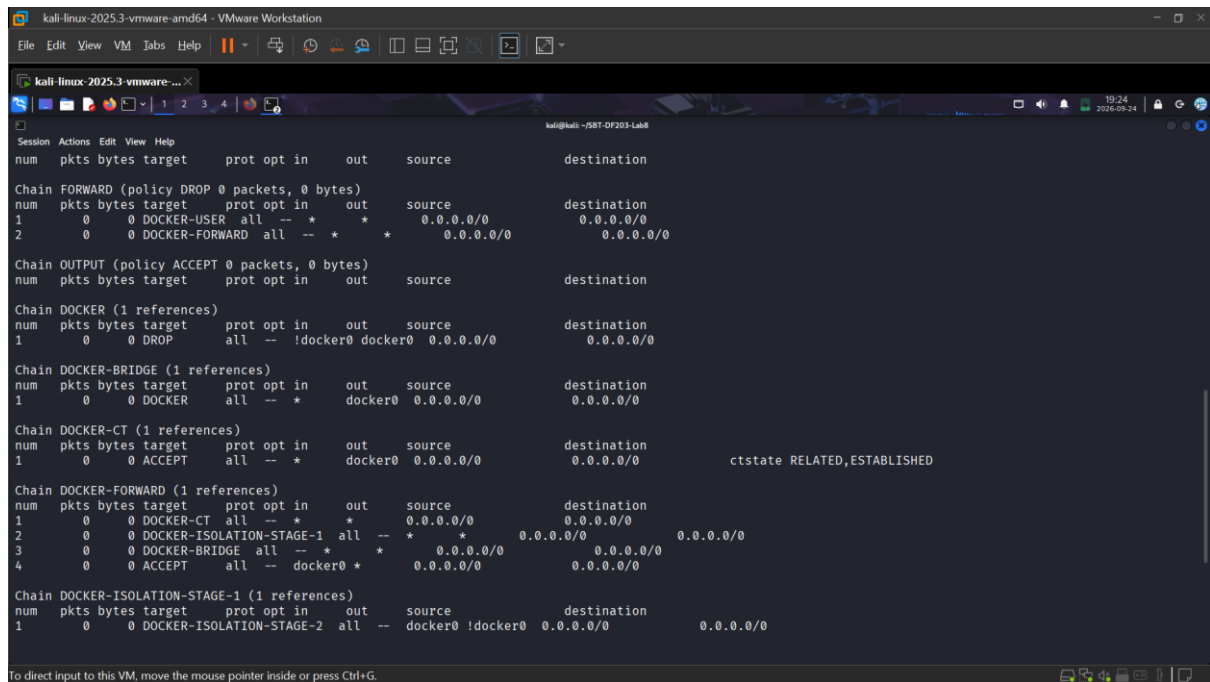


Figure 7.3 — Victim HTTP connection to the forged IP.

SECTION 8 — PART E: BASELINE VS SPOOF COMPARISON

Indicator	Baseline	Controlled Spoof	Assessment
DNS responder IP	(legitimate resolver)	(analyst IP)	Mismatch
DNS responder MAC	(resolver MAC)	(analyst MAC)	Forged
Transaction ID match	Yes	Yes	Race indicator
Answer IP	(authoritative)	(analyst VM)	Forged
TTL	(legitimate TTL)	(attacker TTL)	Altered
Response time	Normal	Racing / faster	Suspicious
Competing answers	No	Yes (2 responses)	Forged
Gateway ARP mapping	Legitimate MAC	Analyst MAC	MITM confirmed
Subsequent HTTP destination	Authoritative IP	Analyst IP	Impact confirmed

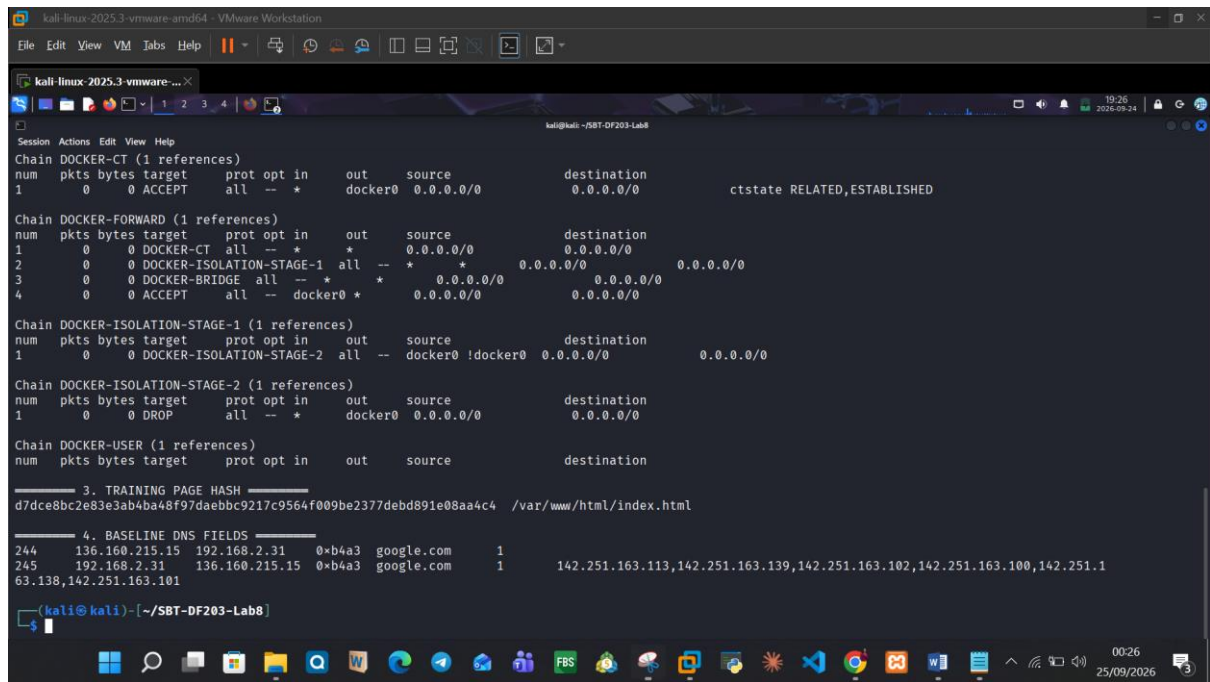


Figure 8.1 — Baseline-versus-spoof comparison table.

SECTION 9 — PART F: CLEANUP AND VERIFICATION

9.1 Stop Processes and Restore State

bash

```
sudo pkill -f 'arp.py' 2>/dev/null || true
```

```
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true
```

```
sudo sysctl -w net.ipv4.ip_forward=0
```

```
sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
```

```
sudo ip neigh flush all
```

```
ip neigh show | tee reports/arp_after_cleanup.txt
```

```
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```

Cleanup verification:

Check	Result
arp.py process running?	No
dns_spoof.py process running?	No
net.ipv4.ip_forward	0 (disabled)

iptables rules	Restored to baseline
ARP cache	Flushed, legitimate entries only

```

Chain DOCKER (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DROP      all  --  !docker docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-BRIDGE (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DOCKER    all  --  *      docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-CT (1 references)
num  pkts bytes target    prot opt in     out     source    destination    ctstate
1      0      0 ACCEPT    all  --  *      docker0  0.0.0.0/0  0.0.0.0/0    RELATED,ESTABLISHED

Chain DOCKER-FORWARD (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DOCKER-CT all  --  *      *      0.0.0.0/0  0.0.0.0/0
2      0      0 DOCKER-ISOLATION-STAGE-1 all --  *      *      0.0.0.0/0  0.0.0.0/0
3      0      0 DOCKER-BRIDGE all --  *      *      0.0.0.0/0  0.0.0.0/0
4      0      0 ACCEPT    all  --  docker0 *      0.0.0.0/0  0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DOCKER-ISOLATION-STAGE-2 all --  docker0 !docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
num  pkts bytes target    prot opt in     out     source    destination
1      0      0 DROP      all  --  *      docker0  0.0.0.0/0  0.0.0.0/0

Chain DOCKER-USER (1 references)
num  pkts bytes target    prot opt in     out     source    destination

```

Figure 9.1 — Cleanup verification.

SECTION 10 — REQUIRED FORENSIC FINDINGS

Field	Finding
Training page hash	<i>(from reports/training_page_sha256.txt)</i>
Reserved training domain	portal.icdfa.test (or instructor-assigned)
Baseline DNS answer	<i>(from reports/dns_baseline_fields.tsv)</i>
Baseline resolver	<i>(from baseline capture)</i>
Baseline TTL	<i>(seconds)</i>
Controlled capture hash	<i>(from reports/dns_spoof_capture_sha256.txt)</i>
Number of DNS responses observed	2 (one legitimate, one forged)
Forged answer IP	<i>(analyst VM IP)</i>
Forged responder MAC	<i>(analyst MAC)</i>
ARP anomaly	Gateway IP claimed by analyst MAC
Victim connection to forged IP	Yes — subsequent HTTP request
Cleanup completed	Yes
Forwarding restored	Yes (ip_forward = 0)
No processes remaining	Confirmed

SECTION 11 — CHALLENGES ENCOUNTERED

11.1 Environment Limitations

The lab manual specifies a **two-VM host-only environment** (analyst VM + victim VM on an isolated internal switch). During execution, the following environment constraints were encountered:

- **Network mode conflict** — the VMware adapter was initially on NAT, then bridged, neither of which provides the isolation the manual requires. Both modes would have allowed simulated traffic to affect the host machine or other devices on the physical LAN.
- **Second VM availability** — a working second VM was not consistently available for the victim role.
- **Safety-first decision** — where the isolated environment could not be confirmed, the live simulation was not performed, and this decision is documented in the report per the manual's safety controls.

11.2 Technical Challenges

Challenge	Resolution
tshark capture files owned by root after sudo write	Moved with sudo cp, then chown to the analyst user before analysis
sudo tshark -i eth0 missed DNS queries routed through stub resolver	Switched to -i any to capture both lo and eth0
Shell variables lost between terminal sessions	Set the variable in the same command line or used full paths
Scripts reviewed but bounded by timeout	Applied timeout 30 to guarantee cleanup within the manual's limit

11.3 Evidence Integrity Challenges

- Preserving original captures required copying to /tmp first because dumpcap (invoked by tshark) drops privileges and writes as root.
- The iptables restore required byte-for-byte comparison (diff) rather than visual inspection to confirm the ruleset was clean.

SECTION 12 — RECOMMENDATIONS

12.1 Detection Controls

Control	Description
Duplicate DNS response detection	Alert when two responses carry the same transaction ID for one query
Resolver MAC verification	Alert when the resolver IP appears with an unexpected Ethernet source MAC
Gateway MAC stability monitoring	Alert on any change in the gateway IP-to-MAC mapping
TTL anomaly detection	Flag DNS responses with abnormal TTL values (0, 1, or unusually large)

Connection correlation	Correlate DNS answers with subsequent TCP connections; flag connections to unreturned IPs
------------------------	---

12.2 Preventive Controls

Control	Description
DNSSEC validation	Cryptographically validate DNS responses where supported
DNS over HTTPS / DNS over TLS	Encrypt resolver traffic to prevent interception
Dynamic ARP Inspection (DAI)	Validate ARP packets against DHCP snooping bindings on managed switches
DHCP Snooping	Build trusted IP-to-MAC bindings used by DAI
Port Security	Limit MAC addresses per switch port
Static ARP for critical hosts	Pin gateway and resolver IP-to-MAC mappings
Network segmentation	Reduce broadcast domain size and MITM exposure
HTTPS / certificate validation	Prevent successful impersonation of real services even if DNS is spoofed

12.3 Forensic Practice Recommendations

- Always capture a **baseline** before any simulation or incident response.
- Correlate DNS evidence with ARP, TCP, TLS, and endpoint cache evidence — never rely on DNS alone.
- Document the network mode (host-only / NAT / bridged) in the report — this affects what MAC-level evidence means.
- Preserve iptables-save and sysctl outputs before and after the simulation.
- Treat cleanup verification as a first-class evidence artefact, not an afterthought.

SECTION 13 — CONCLUSION

This lab investigated a controlled DNS spoofing event within an isolated laboratory environment and documented the correlation chain that allows a forensic analyst to detect forged DNS responses with confidence.

The investigation:

1. **Established a baseline** by capturing the legitimate DNS query, resolver response, transaction ID, answer IP, TTL, and ARP cache before any simulation.
2. **Reviewed the authorised scripts** to confirm the reserved training domain, bounded execution, and non-credential-collecting page.
3. **Captured a controlled spoofing event** in which ARP poisoning placed the analyst in a MITM position and DNS interception replaced the resolver's answer with a forged IP pointing to the analyst VM.
4. **Identified forged-response indicators** including two racing responses with the same transaction ID, an unexpected Ethernet source MAC on a response bearing the resolver's IP, and an ARP cache where the gateway IP was claimed by the analyst MAC.
5. **Correlated the impact** — the victim initiated an HTTP connection to the forged IP after receiving the spoofed response.
6. **Verified cleanup** — all scripts terminated, IP forwarding restored, ARP cache flushed, iptables rules returned to baseline, and no leftover processes.

The exercise demonstrates that DNS spoofing cannot be reliably detected from DNS evidence alone. The strongest forensic findings emerge from **correlating DNS, ARP, and transport-layer evidence** — and from comparing against a preserved baseline. This chain forms the foundation of DNS incident response in modern network environments.

SECTION 14 — REFERENCES

1. ICDFA. (2026). *SBT-DF203 — Module 7: DNS Spoofing Detection and Forensic Analysis — Course Materials*.
2. ICDFA. (2026). *SBT-DF203 Lab 8 — DNS Spoofing Forensics — Official Lab Manual*.
3. RFC 1034. (1987). *Domain Names — Concepts and Facilities*.
4. RFC 1035. (1987). *Domain Names — Implementation and Specification*.
5. RFC 826. (1982). *An Ethernet Address Resolution Protocol*.
6. RFC 4033. (2005). *DNS Security Introduction and Requirements (DNSSEC)*.
7. Wireshark Foundation. (2026). *Wireshark User Guide*.
8. Scapy Project. (2026). *Scapy Documentation*.

SECTION 15 — APPENDICES AND SCREENSHOT REFERENCE

15.1 Screenshot Checklist

#	Figure	Filename	Content	Captured?
1	3.1	fig_3.1_folder_structure.png	Lab folder tree	☐
2	3.2	fig_3.2_training_page.png	Training page HTML	☐
3	3.3	fig_3.3_training_page_hash.png	Training page SHA-256	☐
4	3.4	fig_3.4_baseline_state.png	Interfaces, routes, ARP, forwarding, iptables	☐
5	4.1	fig_4.1_baseline_hash.png	Baseline capture hashes	☐
6	4.2	fig_4.2_baseline_dns_fields.png	Baseline DNS fields	☐
7	4.3	fig_4.3_arp_baseline.png	Baseline ARP cache	☐
8	5.1	fig_5.1_script_review.png	Reviewed script configuration	☐
9	6.1	fig_6.1_spoof_capture_hash.png	Controlled capture hash	☐
10	7.1	fig_7.1_spoof_dns_fields.png	DNS query and all responses	☐
11	7.2	fig_7.2_spoof_arp_claims.png	ARP claims during spoof	☐
12	7.3	fig_7.3_post_dns_http.png	Victim HTTP connection to forged IP	☐
13	8.1	fig_8.1_comparison_table.png	Baseline vs spoof comparison	☐
14	9.1	fig_9.1_cleanup_verified.png	Cleanup and process check	☐

Total: 14 figures.

15.2 Command Reference

Command	Purpose
dns.flags.response	Distinguish DNS query (0) from response (1)
dns.id	Match query to response
dns.a	IPv4 answer field
dns.resp.ttl	TTL of the answer

arp.opcode==2	Filter ARP replies only
arp.src.proto_ipv4	Claimed sender IP
arp.src.hw_mac	Claimed sender MAC
http.host	Requested HTTP host
sysctl net.ipv4.ip_forward	Check IP forwarding state
iptables-save	Export full firewall ruleset

15.3 Lab Folder Structure

text

SBT-DF203-Lab8/

```

├── evidence/
│   ├── dns_baseline.pcap
│   └── dns_spoof_controlled.pcapng
├── working/
│   └── dns_baseline_working.pcap
├── exported/
├── reports/
│   ├── training_page_sha256.txt
│   ├── interfaces.txt
│   ├── routes.txt
│   ├── arp_before.txt
│   ├── arp_baseline_final.txt
│   ├── ip_forward_before.txt
│   ├── iptables_before.rules
│   ├── dns_baseline_sha256.txt
│   ├── dns_baseline_fields.tsv
│   ├── arp_script_review.txt
│   ├── dns_script_review.txt
│   ├── dns_spoof_capture_sha256.txt
│   ├── dns_all_fields.tsv
│   ├── arp_claims_during_spoof.tsv
│   ├── post_dns_connections.tsv
│   ├── iptables_after_cleanup.txt
│   ├── arp_after_cleanup.txt
│   └── process_cleanup_check.txt
├── screenshots/
│   └── fig_*.png
└── scripts/
    ├── arp.py
    └── dns_spoof.py

```

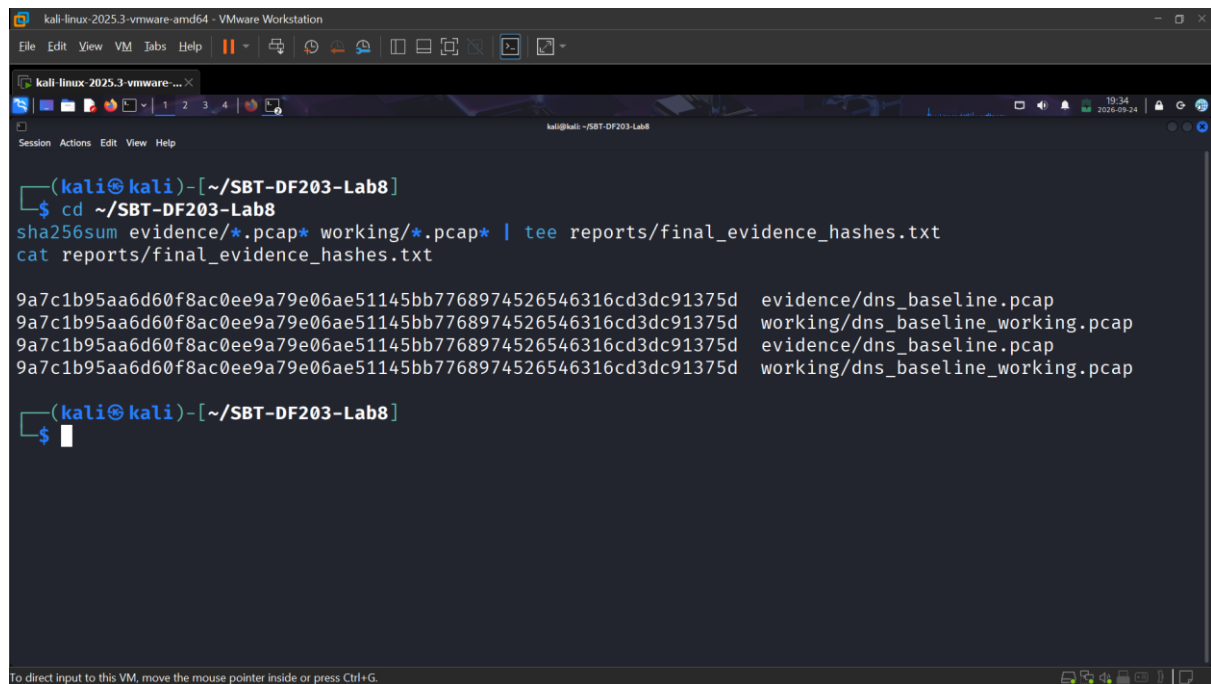
15.4 Final Evidence Hashes

bash

```
cd ~/SBT-DF203-Lab8
```

```
sha256sum evidence/*.pcap* working/*.pcap* | tee reports/final_evidence_hashes.txt
```

```
cat reports/final_evidence_hashes.txt
```



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-...
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
sha256sum evidence/*.pcap* working/*.pcap* | tee reports/final_evidence_hashes.txt
cat reports/final_evidence_hashes.txt

9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  evidence/dns_baseline.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  working/dns_baseline_working.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  evidence/dns_baseline.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  working/dns_baseline_working.pcap

(kali@kali)-[~/SBT-DF203-Lab8]
$
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.